You can get more structured output with `console.table`, which takes your arrays, objects, arrays of objects, or objects of arrays and tries to put them in a convenient tabular format in the console. (I need to remember to use this one more—it's quite useful.)

If you'd like a different way to call attention to your message, you can actually embed CSS into the call using string substation and a `%c`, as in `console.log("%c user: %o", "font-size: 24pt; color: green", user)`. Most text-based CSS will work here; again, browsers will vary.

Sometimes all you want to know is whether a particular line is called and you don't really have a message. The `console.count` method just prints out a count, incrementing by one every time it is called. If you give `count` an argument, that argument is displayed with the count as tag, and `count` maintains a separate value for each tag. The method `countReset` resets the count back to zero and may or may not display a `0` message depending on the browser.

Often you only want a message to appear in the console conditionally. The method, `console.assert` takes two arguments: a boolean expression and a string. The method only displays the string if the assertion is false. Again, different browsers will display the assertion failure differently.

Accessing objects in the Rails namespace of your code is a little tricky. I recommend explicitly (and temporarily) assigning an object that you are concerned about to the global object with something like `window.object_under_debug = user`, and then you can access it in the console as `object_under_debug`. If you want to do some DOM manipulation, `$0` provides the object that is currently selected in the DOM element selector part of the browser, and you can also do `$("selector")` as a shortcut for `document.querySelector`.

Using the Cypress Test Runner

Debugging Cypress tests in the Cypress test runner gives you a few additional features than browser tools.